

Path Traversal атаки в Node.js уеб приложение:

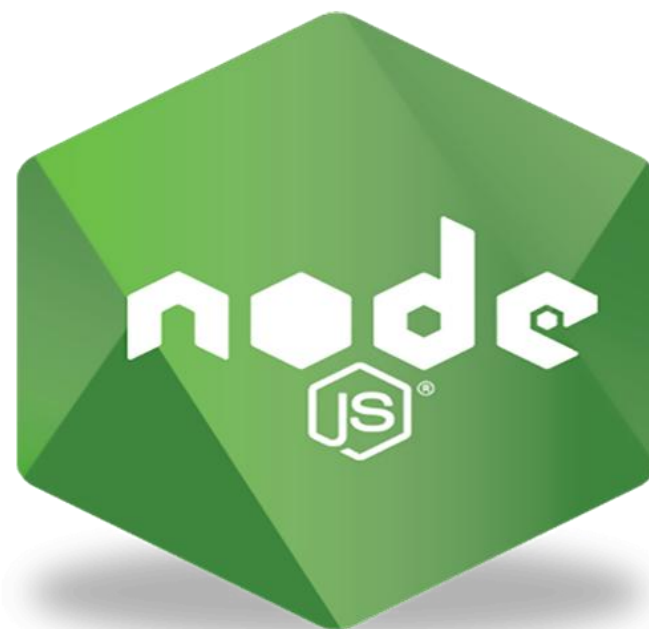
демонстрация, анализ и добри практики за защита

Изготвил: Калоян Механджийски, фн. 5MI0700025

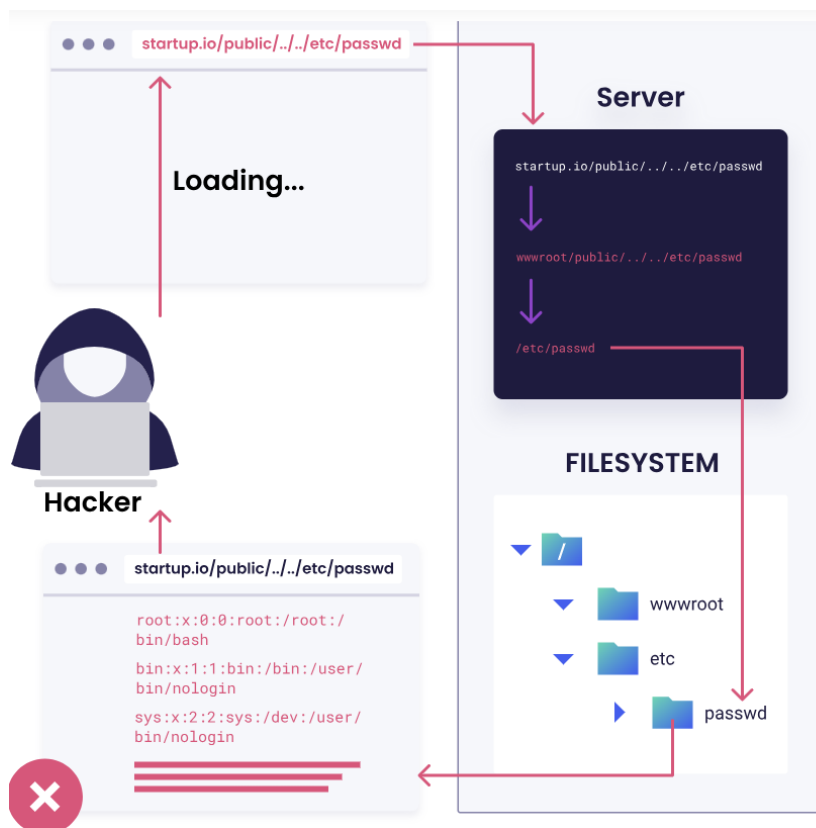


1. Проектът ще демонстрира Path Traversal уязвимост в Node.js/Express приложение.
2. Показва се как потребителски вход, използван директно като файлов път, може да доведе до неоторизиран достъп до файлове.
3. След това е реализирана защитена версия чрез нормализация на пътя и ограничаване до позволена директория.

Used tech stack:



Проблемът: достъп до файлове извън разрешената директория



- Path Traversal възниква, когато уеб приложение използва подаден от потребителя файлов път без достатъчна проверка.
- В Node.js това може да позволи на атакуващ да използва стойности като `../` и да достигне до файлове извън позволената папка.
- Така вместо да се отвори само разрешен документ, приложението може неволно да върне чувствителни файлове от сървъра.

Как да се предпазим?

Защитата срещу Path Traversal се базира на това приложението да не вярва директно на подадения от потребителя път.

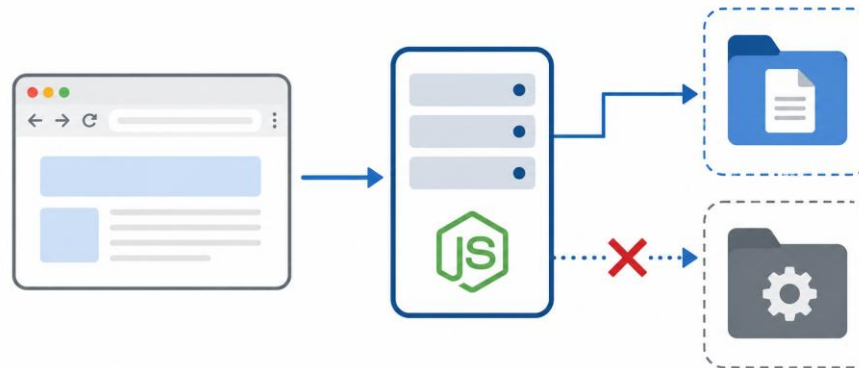
Решението включва:

- нормализиране на пътя;
- проверка дали крайният файл остава в разрешената директория;
- отказване на заявки, които съдържат ../ или водят извън позволената папка;
- използване на allowlist за позволени файлове, когато е възможно.



Как да се предпазим?

Node.js сървърът действа като защитен посредник между потребителя и файловата система.



Той допуска достъп само до файлове от разрешена директория и блокира опити за достъп до системни или външни файлове.

Имплементация:

Към обикновената имплементация endpoint-а `/download-vulnerable`, прибавяме споменатите добри практики от сигурното уеб програмиране и получаваме secure endpoint-а `/download-secure`

```
65 app.get("/download-vulnerable", (req, res) => {
66   const file = req.query.file;
67
68   if (!file) {
69     return res.status(400).send("Missing file parameter.");
70   }
71
72   const unsafePath = path.join(__dirname, "public", file);
73
74   fs.readFile(unsafePath, "utf8", (err, data) => {
75     if (err) {
76       return res.status(404).send("File not found.");
77     }
78
79     res.type("text/plain").send(data);
80   });
81 });
```

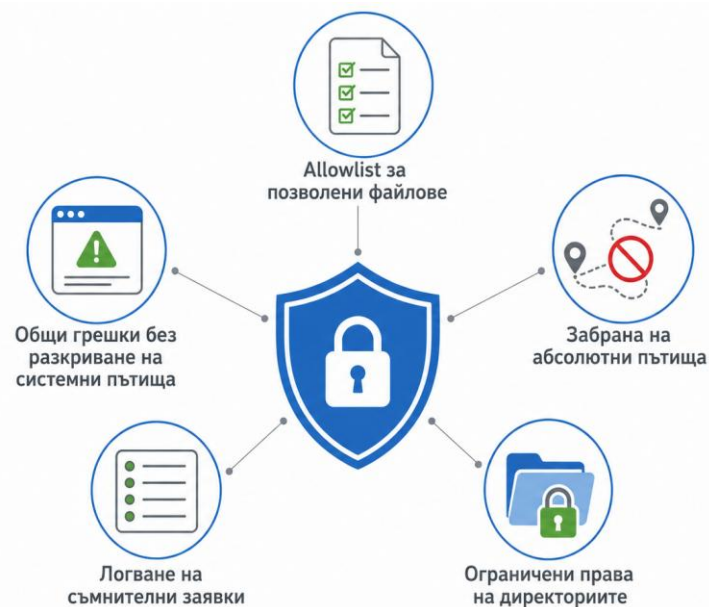
```
93 app.get("/download-secure", (req, res) => {
94   const file = req.query.file;
95
96   if (!file) {
97     return res.status(400).send("Missing file parameter.");
98   }
99
100   // Премахва излишни ".", ".." и повтарящи се наклонени черти.
101   const normalizedFile = path.normalize(file);
102
103   // Създава абсолютен път спрямо разрешената директория.
104   const requestedPath = path.resolve(BASE_DIR, normalizedFile);
105
106   // Гарантира, че requestedPath е вътре в BASE_DIR.
107   // Добавяме path.sep, за да избегнем частични съвпадения като:
108   // /app/public2 да мине като /app/public.
109   const baseWithSeparator = BASE_DIR.endsWith(path.sep) ? BASE_DIR : BASE_DIR + path.sep;
110
111   if (requestedPath !== BASE_DIR && !requestedPath.startsWith(baseWithSeparator)) {
112     console.warn(`[BLOCKED] Path Traversal attempt: ${file}`);
113     return res.status(403).send("Access denied.");
114   }
115
116   fs.readFile(requestedPath, "utf8", (err, data) => {
117     if (err) {
118       return res.status(404).send("File not found.");
119     }
120
121     res.type("text/plain").send(data);
122   });
123 });
```

Идеи за разширение на проекта

Освен проверката на файловия път, приложението трябва да използва и допълнителни защитни механизми. Ето няколко основни, с които проектът може да бъде развит на следващ етап.

Добри практики:

- използване на allowlist за позволени файлове;
- забрана на абсолютни пътища;
- ограничени права на директориите;
- логване на съмнителни заявки;
- връщане на общи грешки без разкриване на системни пътища.



ИЗТОЧНИЦИ:

[1] OWASP – Path Traversal

https://owasp.org/www-community/attacks/Path_Traversal

[2] MITRE CWE-22 – Improper Limitation of a Pathname to a Restricted Directory

<https://cwe.mitre.org/data/definitions/22.html>

[3] Node.js Documentation – Path module

<https://nodejs.org/api/path.html>

[4] Node.js Documentation – File System module

<https://nodejs.org/api/fs.html>

[5] Express.js Documentation

<https://expressjs.com/>

[6] Images

<https://learn.snyk.io/lesson/directory-traversal/?ecosystem=javascript>

[7] Generated images

<https://chatgpt.com/>